06e — OWASP Top 10

Key Concepts

- **OWASP** — Open Web Application Security Project; publishes the 10 most critical web security risks
- **IDOR** — Insecure Direct Object Reference; accessing a resource by ID without ownership check
- **Privilege Escalation** — horizontal (same level, different user) or vertical (higher privilege level)
- **SQL Injection** — user input interpreted as SQL code instead of data
- **Parameterized Query** — SQL structure compiled first, input bound as pure data — injection structurally impossible
- **Salting** — random unique string added to password before hashing to prevent rainbow table attacks
- **SSRF** — Server-Side Request Forgery; server tricked into fetching internal/cloud resources
- **Cloud Metadata Endpoint** — internal IP (169.254.169.254) that returns IAM credentials on AWS/Azure/GCP
- **Rate Limiting** — capping requests per user/IP/endpoint to prevent brute force
- **Credential Stuffing** — using leaked credentials from one breach to attack another service

---

OWASP Top 10 (2021)

#	Name	One-liner
A01	Broken Access Control	User accesses resources they don't own
A02	Cryptographic Failures	Sensitive data exposed via weak/missing encryption
A03	Injection	Malicious input executed as code
A04	Insecure Design	Architecture itself is flawed
A05	Security Misconfiguration	Default passwords, open buckets, verbose errors
A06	Vulnerable & Outdated Components	Libraries with known CVEs
A07	Identification & Authentication Failures	Weak login, no MFA, sessions not invalidated
A08	Software & Data Integrity Failures	Trusting unverified updates or serialized data
A09	Security Logging & Monitoring Failures	Can't detect or respond to breaches
A10	SSRF	Server tricked into fetching internal resources

How Each Key Vulnerability Works

A01 — Broken Access Control

1. User authenticated (logged in)
2. User requests resource by ID (`GET /orders/1042`)
3. Server checks auth but NOT ownership
4. Any user can access any resource

Fix: Server-side ownership check on every request. Return 404 (not 403) to avoid confirming resource existence.

A03 — Injection (SQL)

1. App concatenates user input into SQL string
2. Attacker inputs `' OR '1'='1`
3. Database executes manipulated query
4. Returns all rows / bypasses auth

Fix: Parameterized queries — SQL compiled first, input bound as data. Never concatenate user input.

A07 — Auth Failures

1. No rate limiting → brute force possible
2. Sessions not invalidated server-side on logout → stolen tokens remain valid
3. No MFA → single factor compromised = full access

Fix: Rate limit logins, server-side session invalidation, MFA on sensitive operations.

A10 — SSRF

1. App fetches a URL on user's behalf
2. Attacker passes internal URL (`http://169.254.169.254/...`)
3. Server (inside the network) fetches it
4. Returns IAM credentials / internal data to attacker

Fix: Whitelist allowed domains, block private IP ranges, network-level firewall on outbound requests.

Industry Examples

Company	Vulnerability	Impact
Venmo (2019)	A01 Broken Access Control	200M transactions scraped via public API
Capital One (2019)	A10 SSRF	100M records, AWS creds stolen via metadata endpoint, $80M fine
LinkedIn (2012)	A07 Auth + A02 Crypto	6.5M unsalted SHA1 hashes cracked immediately
Heartland (2009)	A03 SQL Injection	130M credit card numbers stolen
Mirai Botnet (2016)	A07 Default Credentials	600K IoT devices compromised, took down Twitter/Netflix/Reddit
Dropbox (2016)	A07 Credential Stuffing	68M credentials exposed

Interview Questions

Q: What is IDOR and how do you prevent it? A: IDOR is accessing a resource by ID without verifying ownership. Fix: server-side ownership check + return 404 not 403.

Q: Why return 404 instead of 403 on unauthorized access? A: 403 confirms the resource exists — information leakage. 404 reveals nothing, preventing enumeration.

Q: Why is parameterized query safer than sanitization? A: Parameterization is structural — input can never become code. Sanitization is a blacklist — attackers find encoding tricks to bypass it.

Q: What is SSRF and why is it dangerous in cloud? A: Server fetches attacker-controlled URLs, hitting internal endpoints. Cloud metadata at 169.254.169.254 returns IAM credentials — full cloud takeover possible.

Q: What is credential stuffing? A: Using leaked username/password pairs from one breach to attack other services, exploiting password reuse.